



PROYECTO FINAL (AE3) — CLÚSTER MPI Y MINERÍA DISTRIBUIDA

HITO 3 — Paralelización con MPI

Grupo

Asignatura

ESTRUCTURA DE COMPUTADORES, PARALELISMO Y SISTEMAS DISTRIBUIDOS
(GRADO EN CIBERSEGURIDAD)

1. Objetivo del Hito En este tercer hito pasaremos a una ejecución multiproceso dentro de la misma máquina virtual. Aprenderéis a:

- Instalar y configurar el entorno necesario para la interfaz de paso de mensajes (MPI).
- Entender los conceptos básicos de mpi4py (Comm, rank, size).
- Modificar vuestro minero para que múltiples procesos dividan el trabajo de forma estática.

En base al minero secuencial desarrollado en el Hito 1, en este hito modificaréis su arquitectura para permitir una ejecución multiproceso mediante MPI.

2. Contexto

Como demostrasteis en el Hito 2, el rendimiento del minero secuencial está limitado por la capacidad de un único núcleo de la CPU, ya que el cálculo del hash SHA-256 acapara casi todo el tiempo de procesamiento. Para superar esta barrera, introduciremos la librería mpi4py. Esto nos permitirá lanzar múltiples procesos en paralelo; cada proceso se ejecutará en un núcleo distinto y colaborarán para encontrar el *nonce* válido dividiéndose el espacio de búsqueda.

3. Parte I — Preparación del Entorno MPI

Antes de programar, es necesario dotar a la máquina virtual (Nodo 0 / Master) de las herramientas subyacentes. MPI (Message Passing Interface) no es nativo de Python, requiere un motor a nivel de sistema operativo.

1. Instalar el motor OpenMPI en Ubuntu:

Abrid la terminal de vuestra máquina virtual y ejecutad:

```
sudo apt update
```

```
sudo apt install -y openmpi-bin libopenmpi-dev
```

2. Instalar la librería de Python mpi4py: Una vez instalado el motor del sistema, instalamos el *wrapper* para Python:

```
pip3 install mpi4py
```



PROYECTO FINAL (AE3) — CLÚSTER MPI Y MINERÍA DISTRIBUIDA

HITO 3 — Paralelización con MPI

Grupo

Asignatura

ESTRUCTURA DE COMPUTADORES, PARALELISMO Y SISTEMAS DISTRIBUIDOS
(GRADO EN CIBERSEGURIDAD)

4. Parte II — Conceptos Básicos de mpi4py

Para paralelizar el código, debéis importar la librería en vuestro script de Python (from mpi4py import MPI). MPI crea un "comunicador global" donde agrupa a todos los procesos que se lanzan a la vez. Necesitaréis usar dos variables fundamentales:

- **comm = MPI.COMM_WORLD**: Es el grupo de todos los procesos.
- **size = comm.Get_size()**: Nos dice **cuántos** procesos totales se están ejecutando (si lanzáis el programa con 4 procesos, size será 4).
- **rank = comm.Get_rank()**: Nos dice **cuál** es el proceso actual. Va de 0 hasta size - 1. Es el identificador único de cada trabajador.

5. Parte III — Paralelización estática del minero Basándonos en el código secuencial del Hito 1, cread un nuevo script llamado `minero_mpi_basico.py`.

El objetivo principal es evitar que todos los núcleos calculen los mismos hashes. Debéis modificar el bucle for para que Múltiples procesos dividan el trabajo.

Si tenemos 2 procesos (size = 2):

- El proceso rank 0 prueba los *nonces* pares (0, 2, 4, 6...).
- El proceso rank 1 prueba los impares (1, 3, 5, 7...).

Para implementar esto, usad la fórmula $\text{nonce} = \text{rank} + i * \text{size}$.

Pista: En lugar de usar la fórmula manualmente, la función `range()` de Python os permite hacer esto de forma nativa utilizando su tercer parámetro (el "salto" o *step*): `range(inicio, fin, salto)`. Pensad cómo encajan rank y size en esa función.

for nonce in range(rank, max_nonce, size):

6. Parte IV — Ejecución y Pruebas (el comando mpirun)

Los scripts de MPI no se ejecutan con `python3 script.py` directamente. Se utiliza el comando `mpirun` o `mpiexec` para indicar cuántos procesos queremos lanzar.

1. Abrid una terminal y lanzad vuestro nuevo minero con 2 procesos:

```
mpirun -n 2 python3 minero_mpi_basico.py --dificultad 4 --max 2000000
```



PROYECTO FINAL (AE3) — CLÚSTER MPI Y MINERÍA DISTRIBUIDA

HITO 3 — Paralelización con MPI

Grupo

Asignatura

ESTRUCTURA DE COMPUTADORES, PARALELISMO Y SISTEMAS DISTRIBUIDOS
(GRADO EN CIBERSEGURIDAD)

2. Mientras se ejecuta, abrid otra terminal y lanzad `htop`.
3. **Comprobación:** Deberíais ver que ahora **dos** barras de vuestra CPU se ponen al 100%, y en la lista de procesos aparecerán dos instancias de Python corriendo simultáneamente.

Cuando ejecutamos un programa con *mpirun*, se lanzan múltiples procesos independientes del mismo script, cada uno con su propio espacio de memoria.

7. Entregables

El grupo debe entregar en un único fichero comprimido ZIP:

- El script `minero_mpi_basico.py` corriendo en varios núcleos de la VM local.
- Captura de pantalla de la terminal ejecutando el minero con `mpirun` resolviendo el bloque.
- Captura de pantalla de `htop` (tomada simultáneamente) demostrando que se están utilizando múltiples núcleos al 100%.

8. Preparación para el Hito 4 Al finalizar este hito tendréis un minero que aprovecha toda la CPU local. Sin embargo, notaréis un problema crítico: si el proceso rank 1 encuentra la solución, el proceso rank 0 seguirá calculando hashes ciegamente hasta agotar su bucle `range`. En el siguiente hito, resolveremos el mayor desafío de la minería paralela implementando comunicación no bloqueante para que el nodo *Master* avise a los demás y terminen limpiamente todos sus procesos.